

Barber Booking System — Full Project Documentation

1. Overview

The **Barber Booking System** is a full-stack web application designed to streamline appointment scheduling for barbershops.

It provides a seamless experience for **customers**, **barbers**, and **administrators**, offering real-time availability, secure authentication, and intuitive management tools.

The system is built using:

- **ASP.NET Core Web API** (backend)
- **Entity Framework Core** (database)
- **React + Vite** (frontend)
- **TailwindCSS** (UI)
- **JWT Authentication** (security)

2. Core Features

Customer Features

- Create an account & log in securely
- Browse services
- Select barber, date, and time
- View real-time availability
- Book appointments
- Cancel upcoming bookings
- View booking history

Barber Features

- Manage personal working hours
- View upcoming appointments
- Prevent double-booking
- Update availability

Admin Features

- Manage barbers
- Manage services
- Manage working hours
- View all bookings

- Full CRUD operations

3. System Architecture

Code

Frontend (React + Vite)

|
| authFetch() → attaches JWT
|

Backend (ASP.NET Core Web API)

|
| EF Core
|

Database (SQL Server)

Frontend

- React (Vite)
- React Router
- Context API for authentication
- TailwindCSS for UI
- API_BASE for all API calls

Backend

- ASP.NET Core Web API
- Entity Framework Core
- SQL Server
- Repository + Service pattern
- JWT Authentication

4. Authentication Flow

Login

1. User submits email + password
2. Backend validates credentials
3. Backend returns JWT
4. Frontend stores token in `localStorage`
5. `AuthContext` decodes token
6. User is set globally
7. `authFetch` attaches token to every request

Token Structure

The JWT contains:

- `id`
- `email`
- `role`
- `barberId` (only for barbers)
- `fullName`

Protected Fetch

```
js
authFetch(`${API_BASE}/api/bookings/user/${user.id}`)
```

5. API Structure

Authentication

Code
 POST /api/Auth/login
 POST /api/Auth/register

Bookings

Code
 GET /api/bookings/user/{id}
 POST /api/bookings
 DELETE /api/bookings/{id}

Working Hours

Code
 GET /api/workinghours/{barberId}
 GET /api/workinghours/barber/{id}?date=YYYY-MM-DD&serviceId=X&stepMinutes=15
 POST /api/workinghours
 DELETE /api/workinghours/{id}

Services

Code
 GET /api/services

Barbers

Code
 GET /api/barbers

6. Database Schema

Tables

- Users
- Barbers
- Services
- WorkingHours
- Bookings

Booking Table

Column	Type	Notes
BookingId	int	PK
UserId	int	FK → Users
BarberId	int	FK → Barbers
ServiceId	int	FK → Services
Start	datetime	Appointment start time
End	datetime	Calculated from service

7. Frontend Structure

Code

```
src/  
  api/  
    config.js  
    workingHours.js  
    bookings.js  
  context/  
    AuthContext.jsx  
  pages/  
    Login.jsx  
    Dashboard.jsx  
    Booking/  
    Admin/  
  components/  
    App.jsx  
    main.jsx
```

Key Frontend Concepts

- `AuthContext` handles login, logout, token decoding, and protected fetch.
- `authFetch` automatically attaches JWT.
- `API_BASE` ensures consistent API URLs.
- Booking flow uses real-time availability from the backend.

8. Backend Structure

Code
Controllers/
Models/
DTOs/
Services/
Repositories/
Data/
Program.cs

Key Backend Concepts

- Clean separation of concerns
- DTOs for request/response
- Services for business logic
- EF Core for database access
- JWT middleware for authentication

9. Installation & Setup

Backend

```
bash
cd barber-backend
dotnet restore
dotnet ef database update
dotnet run
```

Backend runs at:

Code
`http://localhost:5078`

Frontend

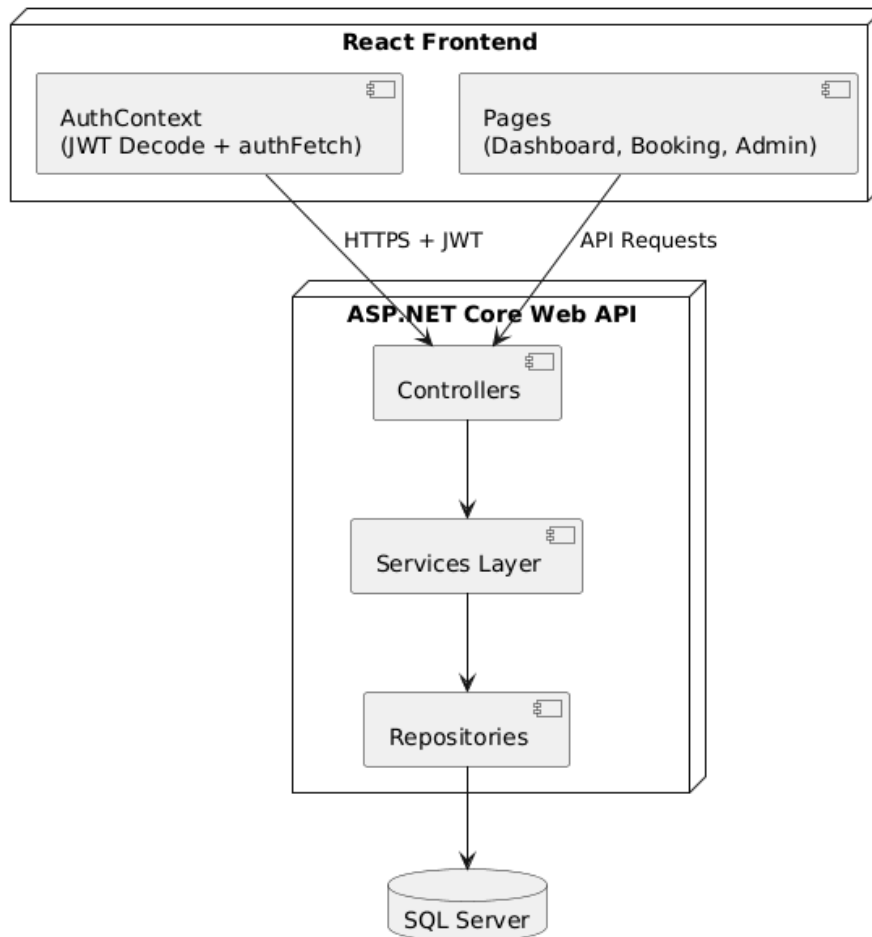
```
bash
cd barber-frontend
npm install
npm run dev
```

Frontend runs at:

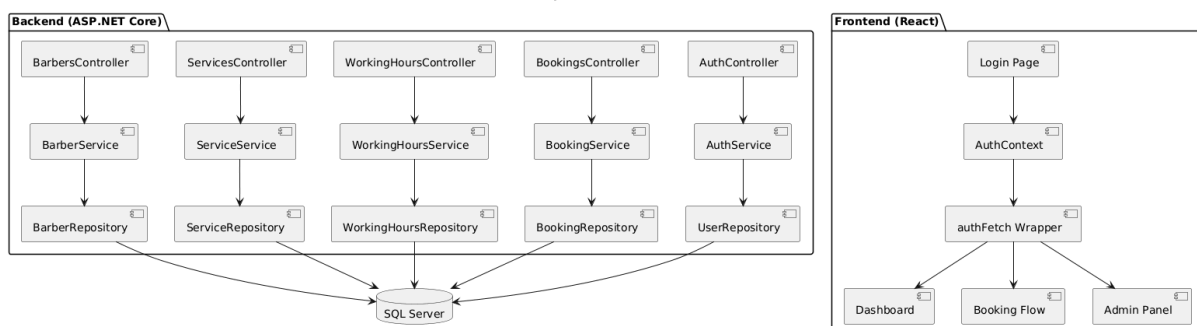
Code

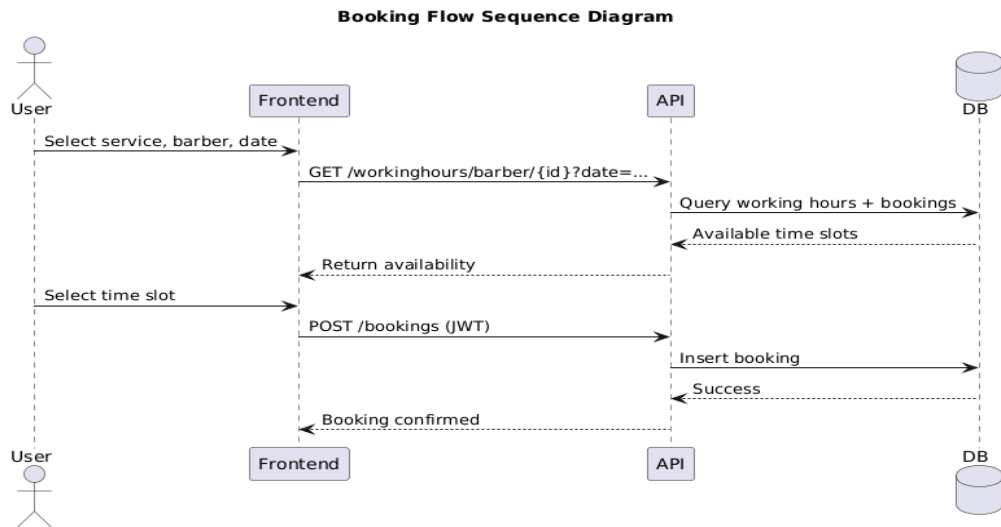
<http://localhost:5173>

System Architecture - Barber Booking System

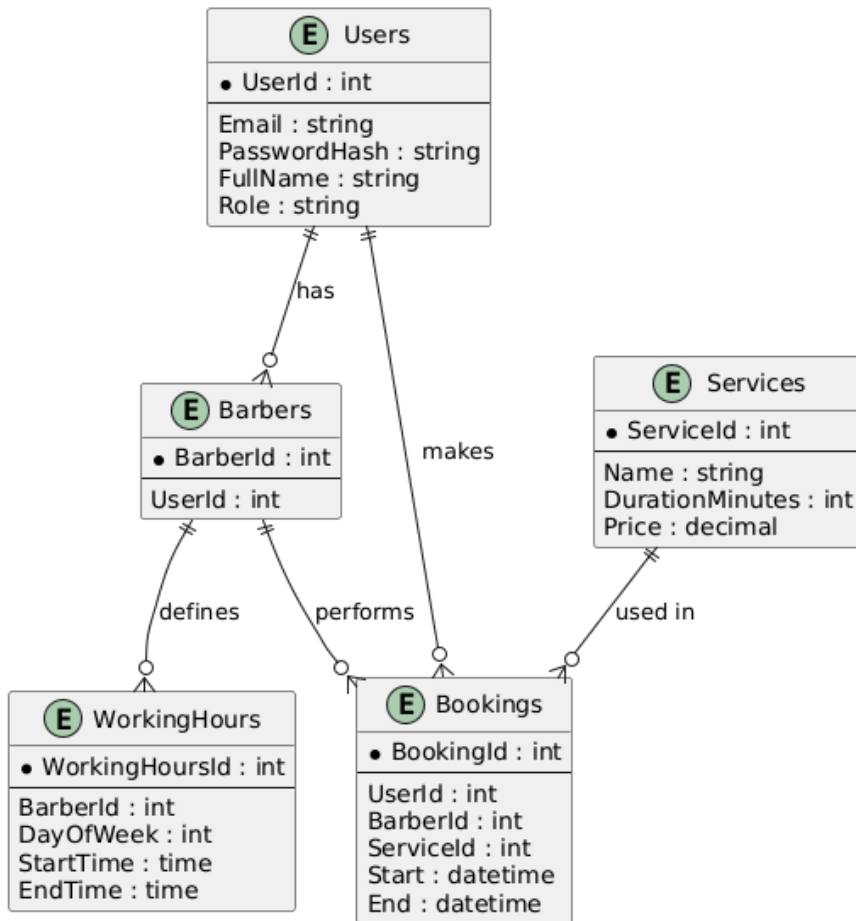


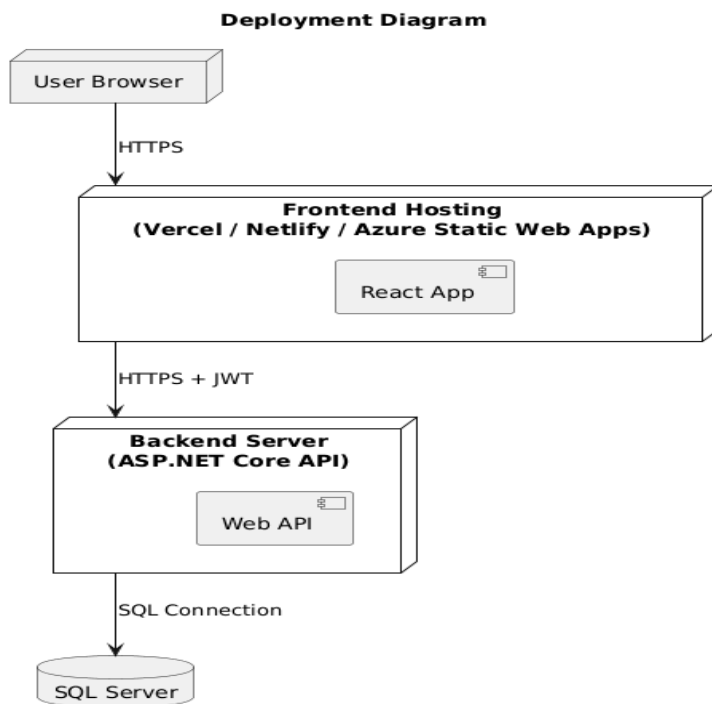
Component Architecture





Entity Relationship Diagram (ERD)





10. Deployment

Backend

- Deploy to Azure App Service or Docker container
- Use SQL Azure or hosted SQL Server

Frontend

- Deploy to Vercel, Netlify, or Azure Static Web Apps

Set environment variable:

Code

VITE_API_BASE=https://your-backend-url

-

11. Future Improvements

- Email notifications for bookings
- Barber profile pages
- Payment integration
- Multi-location support

- Calendar sync (Google/Outlook)
- Mobile app version

12. License

This project is licensed under the MIT License.

13. Author

Irene

Full-stack Software Engineer

Helsingborg, Sweden